LinkedIn Job Analytics Dashboard

Aryan Mittal, Jaskirat Singh, Jayanth Pasupulati
Github: https://github.com/theonlyjaypas/linkedinjobpostings
DATA 201 — Group 2

I. INTRODUCTION

The modern job market generates an enormous volume of data. LinkedIn alone hosts hundreds of thousands of active job postings at any given time, each carrying information about required skills, salary ranges, industries, experience levels, and work arrangements. Yet despite this abundance of data, job seekers typically navigate it through keyword searches and manual browsing: a slow, fragmented process that makes it difficult to spot broader patterns or make informed career decisions.

This project addresses that gap by building an end-to-end analytics system on top of a real-world LinkedIn job postings dataset. The result is a queryable MySQL relational database and an interactive, multi-page web dashboard that transforms raw CSV files into actionable job market insights, with no SQL knowledge required from the end user.

II. PROBLEM STATEMENT

Job seekers face significant information overload. With over 100,000 postings spread across dozens of industries and hundreds of skill categories, there is no easy way to compare salaries, identify in-demand skills, or understand which companies and industries are actively hiring. Existing tools answer surface-level questions but fail to address deeper, data-driven inquiries such as which skills pay the most, which industries are hiring, and whether remote work commands a salary premium.

The raw LinkedIn data exists but is scattered across 11 separate CSV files with no relational structure, making it impossible to analyze without first building a proper database. The goal of this project is to design and implement a normalized MySQL relational database and an interactive Streamlit dashboard that answers 12 distinct analytical questions from a single unified data source, enabling any user to explore the job market in under 60 seconds without writing SQL.

Success Criteria

A working ETL pipeline that loads and normalizes all 11 source CSV files into a clean relational schema, along with twelve analytical query functions each answering one distinct business question, formed the technical core of the project. These were complemented by an interactive dashboard featuring at least six pages with live chart rendering, as well as a job search page that queries the database dynamically based on user input.

III. DATASET

The dataset is a publicly available LinkedIn Job Postings collection sourced from Kaggle (arshkon/linkedin-job-postings), covering postings listed between 2023 and 2024. The raw data

arrives as 11 CSV files organized across three directories: data/postings.csv, data/jobs/ (salaries, job skills, job industries, benefits), and data/companies/ and data/mappings/ (lookup tables for industries and skills).

Metric	Count
Job Postings	123,849
Companies	24,473
Industries	388
Unique Skills	35
Salary Records	40,785
Skill-to-Job Links	213,768

Only approximately 33% of postings include salary data (40,785 of 123,849), so all salary analyses are scoped to this subset. Pay periods vary across records and are normalized to an annual USD equivalent during the ETL process. Missing experience levels are filled with the value "Unknown" and excluded from experience-level analyses. Missing remote flags default to 0 (not remote).

IV. SYSTEM ARCHITECTURE

The system follows a classic three-tier architecture: a data pipeline layer, a database layer, and a presentation layer. The tech stack consists of Python and Pandas for data processing, MySQL for storage, SQLAlchemy with mysql-connector-python for querying, Streamlit as the web framework along with CSS for additional webpage design, and Plotly Express and Plotly Graph Objects for charting.

Layer	Technology
Data Processing	Python, Pandas
Database	MySQL
ORM / Query	SQLAlchemy + mysql-connector-python
Web Framework	Streamlit, CSS
Charting	Plotly Express + Plotly Graph Objects

CSV source files feed into load_data.py, which cleans and loads data into the linkedin_jobs MySQL database (10 tables + 1 view). The analyze_data.py module contains 12 SQL query functions that return Pandas DataFrames. The app.py Streamlit application consumes those DataFrames and renders an interactive dashboard served at localhost:8501.

V. DATABASE SCHEMA

The database linkedin_jobs consists of 10 normalized tables and 1 denormalized analytical view.

A. Tables

DIMENSION/LOOKUP TABLES

- *industries*: maps industry_id to industry_name
- *skills*: maps skill_abr (VARCHAR primary key) to skill_name

COMPANY TABLES

- *companies*: core company profiles including name, size bucket (1–7), location, and LinkedIn URL
- *company_industries*: many-to-many mapping of companies to industries
- *employee_counts*: most recent employee and follower counts per company

JOB/POSTING TABLES

- *postings*: core job postings with title, location, work type, experience level, and remote flag
- *salaries*: salary ranges (min, med, max) per posting, plus a normalized_yearly column in annual USD
- *job_skills*: many-to-many mapping of job postings to required skills
- *job_industries*: many-to-many mapping of job postings to industries
- *benefits*: per-posting list of offered benefits; inferred=1 means auto-detected

B. Relationships

RELATIONSHIP	TYPE
postings -> companies	Many-to-one
postings -> salaries	One-to-one
postings <-> skills (via job_skills)	Many-to-many
postings <-> industries (via job_industries)	Many-to-many
companies <-> industries (via company_industries)	Many-to-many

C. Analytics View

The analytics view is a denormalized JOIN across all core tables (postings, companies, salaries, industries, skills), creating a single flat structure for fast dashboard queries. This avoids repeating complex multi-table JOINs in every query function and is recreated at the end of every ETL run.

VI. IMPLEMENTATION

A. ETL Pipeline [load_data.py]

The ETL pipeline follows a strict Extract, Transform, Load order, inserting dimension tables before fact tables to satisfy foreign key constraints.

Extract

All 11 source CSV files are read into Pandas DataFrames using a shared load_csv() helper.

Transform

Rows with null primary keys are dropped during the transformation stage, and salary values are normalized to annual USD using period multipliers: hourly figures are multiplied by 2,080, weekly by 52, bi-weekly by 26, monthly by 12, and yearly values are kept as is. Company IDs with non-numeric values are coerced to null rather than dropped outright, preserving the associated postings. Industry names in company_industries.csv are resolved to IDs via a lookup against the already-loaded industries table. Duplicate rows are then removed from junction tables, and foreign key integrity is validated before any data is loaded into those tables.

Load

Tables are inserted in dependency order using SQLAlchemy's to_sql() with if_exists="replace". Foreign key checks are disabled at the connection level during bulk loads for performance, then primary keys are added via ALTER TABLE after all data is in place. The analytics view is recreated at the end of every ETL run.

B. Analytics Layer [analyze_data.py]

Twelve query functions each answer one business question and return a Pandas DataFrame. All salary queries filter out values below $20,000 or above $500,000 to remove unrealistic outliers, and require a minimum sample size before including a group in results.

#	FUNCTION	QUESTION ADDRESSED
1	*top_skills_by_demand*	Which skills appear in the most job postings?
2	*top_skills_by_salary*	Which skills have the highest average salary?
3	*top_industries_by_hiring*	Which industries post the most jobs?
4	*top_industries_by_salary*	Which industries pay the most on average?
5	*top_companies_by_hiring*	Which companies post the most jobs?
6	*top_companies_by_salary*	Which companies pay the highest average salary?
7	*salary_by_experience*	How does the salary scale with experience level?
8	*salary_by_work_type*	How does salary compare across work types?
9	*top_job_titles_by_salary*	Which job titles pay the most?
10	*best_industry_skill_combos*	Which industry + skill pairings yield the highest salary?
11	*remote_vs_onsite_salary*	Do remote jobs pay more than on-site jobs?
12	*search_jobs*	Find postings matching a keyword, skill, and/or industry

C. Dashboard [app.py]

The Streamlit dashboard renders across 6 pages: Overview (summary statistics), Skills (demand and salary charts), Industries (hiring volume and salary), Companies (top hiring and top-paying), Salary (breakdowns by experience level, work type, and remote vs. on-site), and Job Search (a live search interface that queries the database on every user input with no pre-loaded data). All charts are built with Plotly and rendered inline within the Streamlit layout.

VII. CONCLUSION

A. Key Findings

The analysis revealed several noteworthy patterns in the job market data. Remote roles commanded a significant salary premium, averaging $140,811 per year compared to $113,148 for on-site positions, a difference of $27,663. Experience level proved to be an equally powerful driver of compensation, with executive roles averaging $221,475 annually, approximately 2.7 times the $81,289 average earned by entry-level professionals. Perhaps most surprisingly, demand did not align neatly with pay: while IT emerged as the most in-demand skill by posting volume, Product Management led all categories in average salary at approximately $179,000 per year despite attracting comparatively fewer postings.

B. Key Lessons

Working through this project surfaced several important methodological insights. Real-world data proved to be far messier than expected, as only about a third of job postings included salary information, and pay periods were inconsistently formatted, requiring careful normalization before any meaningful analysis could begin. Structuring the data into a relational database with properly defined tables and foreign keys paid dividends, making it straightforward to answer complex, multi-dimensional questions that would have been cumbersome otherwise. The project also reinforced how effectively SQL and Python complement one another: SQL handled the relational heavy lifting within the database, while Python and Pandas provided the flexibility needed for cleaning, transforming, and passing results to the visualization layer.

C. Future Work

Several directions could meaningfully extend this project. The dashboard should be deployed publicly so it is accessible without requiring a local MySQL setup. Incorporating more recent LinkedIn data beyond the current 2023–2024 window would keep the findings relevant as market conditions evolve. Adding natural language processing capabilities to extract skills and requirements from unstructured job description text would unlock a richer layer of analysis. On the geographic side, building map-based visualizations to display salary and demand by location would add valuable spatial context. Finally, creating role-specific filtered views, such as dedicated lenses for Data Science or Software Engineering, would make the tool more actionable for job seekers targeting particular fields.